Nova Store

PROYECTO ACADÉMICO · 2026

EVIDENCIAS DE CUMPLIMIENTO

SPA con CRUD y carrito de compras

Aplicación de tienda desarrollada con Vue 3, Vue Router y Pinia. Documento de evidencias con capturas de interfaz, código fuente y explicaciones.

Vue 3 · Vue Router · Pinia · Vite · GitHub Pages

A	**Auriculares**	Producto destacado
K	**Teclado KX-75**	CRUD completo
C	**Carrito**	Total en tiempo real

8	5	2	13
productos iniciales	rutas SPA	stores de Pinia	evidencias

ESTUDIANTE
Brandon · @DonLalencio

REPOSITORIO
github.com/DonLalencio/SPA-con-CRUD-y-Pinia-en-Vue-3

SITIO PUBLICADO
donlalencio.github.io/SPA-con-CRUD-y-Pinia-en-Vue-3/

FECHA
2 de noviembre de 2026

DOCUMENTO DE EVIDENCIAS

Resumen y lista de verificación

Nova Store es una Single Page Application de comercio que permite explorar un catálogo, gestionar productos con un CRUD completo y administrar un carrito de compras. Toda la lógica de datos se concentra en los stores de Pinia y la navegación se resuelve con Vue Router.

Rutas de la aplicación

VISTA	URL	DESCRIPCIÓN
Inicio	`/`	Bienvenida y destacados
Productos	`/productos`	Catálogo con búsqueda y filtros
Crear producto	`/productos/crear`	Formulario de registro
Editar producto	`/productos/:id`	Formulario de edición
Carrito	`/carrito`	Resumen y total acumulado

Ficha técnica

Vue 3 con Composition API, Vue Router 4, Pinia 3 y Vite. Estilos CSS propios y despliegue automatizado en GitHub Pages.

Vue 3 · Vue Router · Pinia · Vite · CSS · GitHub Actions

Entregables

Repositorio:
github.com/DonLalencio/SPA-con-CRUD-y-Pinia-en-Vue-3

Sitio:
donlalencio.github.io/SPA-con-CRUD-y-Pinia-en-Vue-3/

Lista de verificación

- ✓ **Uso de Vue Router para navegación**
  Ev. 02 · págs. 5–6
- ✓ **Uso de Pinia para gestionar productos**
  Ev. 03 · págs. 7–8
- ✓ **Uso de Pinia para gestionar el carrito**
  Ev. 04 · págs. 9–10
- ✓ **Uso de v-for para mostrar productos**
  Ev. 05 · págs. 11–12
- ✓ **Uso de v-if para estados de la aplicación**
  Ev. 06 · págs. 13–14
- ✓ **Uso de v-model en formularios**
  Ev. 07 · págs. 15–16
- ✓ **CRUD completo de productos**
  Ev. 08–10 · págs. 17–22
- ✓ **Funcionalidad de carrito de compras**
  Ev. 11 · págs. 23–24

Cada evidencia ocupa dos páginas: una captura de la interfaz y una captura del código fuente. Se respeta el límite de una sola captura por página y cada evidencia incluye título, explicación breve y la especificación que demuestra.

EVIDENCIA 01 · INTERFAZ DE USUARIO

Repositorio en GitHub y publicación en GitHub Pages

El código fuente está disponible en el repositorio público de GitHub y la aplicación se publica en GitHub Pages mediante un workflow de GitHub Actions. El flujo instala las dependencias, ejecuta la compilación de Vite y despliega la carpeta dist; además se genera un archivo 404.html para que las rutas profundas de Vue Router sigan funcionando al recargar la página.

REPOSITORIO
github.com/DonLalencio/SPA-con-CRUD-y-Pinia-en-Vue-3

SITIO PUBLICADO
donlalencio.github.io/SPA-con-CRUD-y-Pinia-en-Vue-3/



Sitio publicado y funcionando en https://donlalencio.github.io/SPA-con-CRUD-y-Pinia-en-Vue-3/

Especificación cumplida
Publicación en GitHub Pages con compilación y despliegue automáticos

EVIDENCIA 01 · CÓDIGO FUENTE

Configuración del despliegue en GitHub Pages

La base de producción coincide con el nombre del repositorio y el workflow automatiza la compilación y el despliegue en GitHub Pages.

Nova Store · Evidencia de código fuente

Vue 3 · Pinia · Vue Router · Vite

vite.config.js — L. 5–12 — BASE

```
// En GitHub Pages el sitio se publica en /<nombre-del-repositorio>/
// En desarrollo se mantiene en la raíz.
export default defineConfig(({ mode }) => ({
  base: mode === 'production' ? '/SPA-con-CRUD-y-Pinia-en-Vue-3/' : '/',
  plugins: [vue()],
  resolve: {
    alias: {
      '@': fileURLToPath(new URL('./src', import.meta.url))
```

.github/workflows/deploy.yml — L. 19–53 — WORKFLOW

```
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Configurar Node
        uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm

      - name: Instalar dependencias
        run: npm ci

      - name: Construir el sitio
        run: npm run build

      - name: Configurar GitHub Pages
        uses: actions/configure-pages@v5

      - name: Subir artefacto
        uses: actions/upload-pages-artifact@v3
        with:
          path: dist

  deploy:
    needs: build
    runs-on: ubuntu-latest
    environment:
      name: github-pages
      url: ${{ steps.deployment.outputs.page_url }}
    steps:
      - name: Desplegar
        id: deployment
        uses: actions/deploy-pages@v4
```

vite.config.js · base de producción y .github/workflows/deploy.yml

EVIDENCIA 02 · INTERFAZ DE USUARIO

Navegación SPA con Vue Router

La aplicación utiliza Vue Router con historial del navegador para ofrecer una experiencia SPA sin recargas. El menú principal enlaza a Inicio, Productos y Carrito, mientras que RouterView renderiza la vista correspondiente; también se definieron la ruta dinámica /productos/:id y una ruta comodín para páginas no encontradas.



Menú de navegación construido con en la página de inicio

Especificación cumplida
Vue Router · cinco rutas SPA y navegación con router-link

EVIDENCIA 02 · CÓDIGO FUENTE

Definición de rutas con Vue Router

Cada ruta apunta a una vista distinta y comparte el mismo layout. La ruta /productos/:id reutiliza el formulario en modo edición mediante props.

Nova Store · Evidencia de código fuente

Vue 3 · Pinia · Vue Router · Vite

src/router/index.js — L. 6–48

```
import NotFoundView from '@/views/NotFoundView.vue'

const routes = [
  {
    path: '/',
    name: 'inicio',
    component: HomeView,
    meta: { titulo: 'Inicio' }
  },
  {
    path: '/productos',
    name: 'productos',
    component: ProductsView,
    meta: { titulo: 'Productos' }
  },
  {
    path: '/productos/crear',
    name: 'producto-crear',
    component: ProductFormView,
    meta: { titulo: 'Crear producto' }
  },
  {
    path: '/productos/:id',
    name: 'producto-editar',
    component: ProductFormView,
    props: true,
    meta: { titulo: 'Editar producto' }
  },
  {
    path: '/carrito',
    name: 'carrito',
    component: CartView,
    meta: { titulo: 'Carrito' }
  },
  {
    path: '/:pathMatch(.*)*',
    name: 'no-encontrada',
    component: NotFoundView,
    meta: { titulo: 'Página no encontrada' }
  }
]

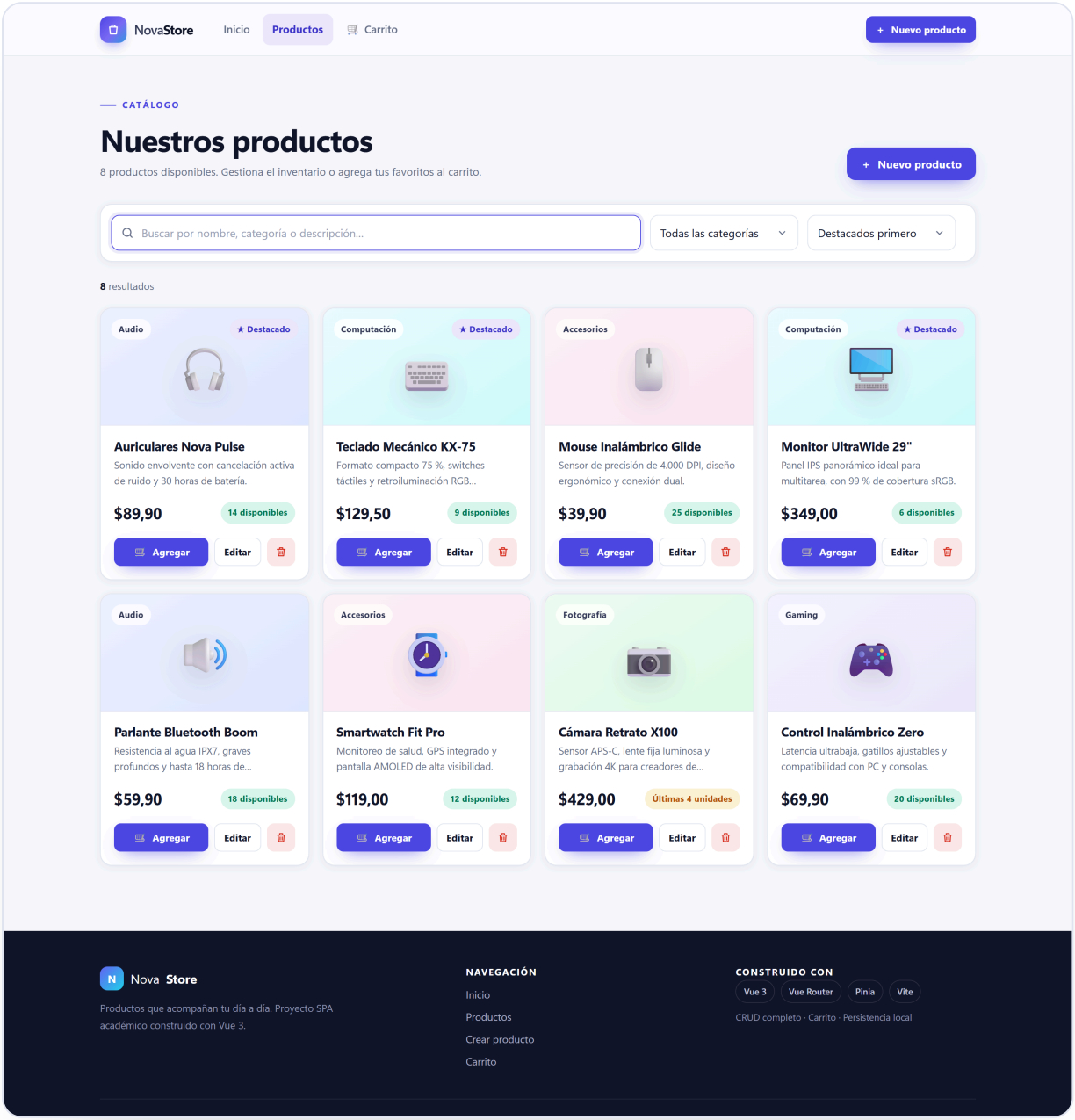
const router = createRouter({
```

src/router/index.js · definición de rutas con createWebHistory

EVIDENCIA 03 · INTERFAZ DE USUARIO

Store de productos con Pinia

El store useProductsStore concentra el estado del catálogo, los getters de consulta y las acciones del CRUD. La vista de productos solo consume los datos y dispara acciones; la búsqueda, los filtros, el ordenamiento, la persistencia en localStorage y la sincronización con el carrito viven en el store, tal como exige la consigna.



Catálogo completo alimentado por el store de productos

Especificación cumplida
Pinia · useProductsStore con estado, getters y CRUD

EVIDENCIA 03 · CÓDIGO FUENTE

Store de productos con Pinia

Se usa un store de composición con defineStore, ref, computed y watch. El estado se hidrata desde localStorage y se guarda automáticamente en cada cambio.

Nova Store · Evidencia de código fuente

Vue 3 · Pinia · Vue Router · Vite

src/stores/products.js — L. 138–178

```
export const useProductsStore = defineStore('productos', () => {
  const productos = ref(cargarProductos())

  watch(
    productos,
    (valor) => {
      localStorage.setItem(CLAVE_PRODUCTOS, JSON.stringify(valor))
    },
    { deep: true }
  )

  const total = computed(() => productos.value.length)
  const destacados = computed(() => productos.value.filter((p) => p.destacado).slice(0, 3))
  const categorias = computed(() => {
    const propias = productos.value.map((p) => p.categoria)
    return [...new Set([...CATEGORIAS, ...propias])].sort((a, b) => a.localeCompare(b, 'es'))
  })

  function obtenerPorId(id) {
    const numero = Number(id)
    return productos.value.find((p) => p.id === numero)
  }

  function buscar({ texto = '', categoria = 'Todas', orden = 'relevancia' } = {}) {
    const termino = sinAcentos(texto)
    let resultado = productos.value.filter((p) => {
      const coincideTexto =
        !termino ||
        sinAcentos(`${p.nombre} ${p.descripcion} ${p.categoria}`).includes(termino)
      const coincideCategoria = categoria === 'Todas' || p.categoria === categoria
      return coincideTexto && coincideCategoria
    })

    resultado = [...resultado]

    if (orden === 'precio-asc') resultado.sort((a, b) => a.precio - b.precio)
    if (orden === 'precio-desc') resultado.sort((a, b) => b.precio - a.precio)
    if (orden === 'nombre') resultado.sort((a, b) => a.nombre.localeCompare(b.nombre, 'es'))

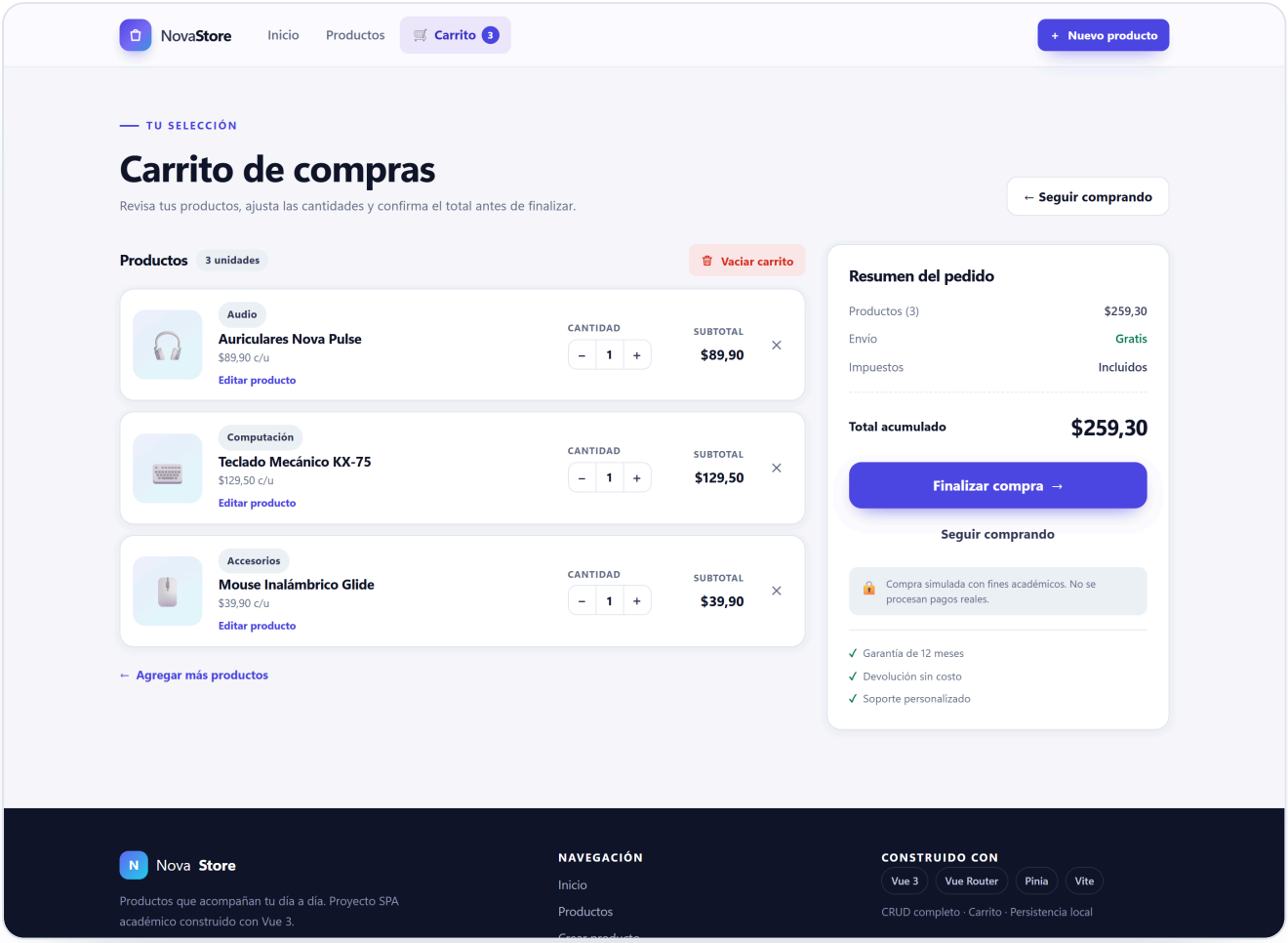
    return resultado
  }
```

src/stores/products.js · estado, getters, búsqueda y persistencia

EVIDENCIA 04 · INTERFAZ DE USUARIO

Store del carrito con Pinia

El store useCartStore administra los productos agregados, las cantidades, los totales y las operaciones de quitar y vaciar. Respeta el stock disponible de cada producto y guarda una instantánea del artículo para conservar el carrito entre sesiones; cuando un producto se edita o elimina, el store de productos lo sincroniza automáticamente.



Carrito con productos, cantidades, subtotales y total acumulado

Especificación cumplida
Pinia · useCartStore con totales y persistencia

EVIDENCIA 04 · CÓDIGO FUENTE

Store del carrito con Pinia

Los totales son getters computados y la acción agregar limita la cantidad al stock. La persistencia se realiza con watch sobre el arreglo de ítems.

Nova Store · Evidencia de código fuente

Vue 3 · Pinia · Vue Router · Vite

src/stores/cart.js — L. 22–65

```
export const useCartStore = defineStore('carrito', () => {
  const items = ref(cargarCarrito())

  watch(
    items,
    (valor) => {
      localStorage.setItem(CLAVE_CARRITO, JSON.stringify(valor))
    },
    { deep: true }
  )

  const estaVacio = computed(() => items.value.length === 0)
  const totalUnidades = computed(() =>
    items.value.reduce((total, item) => total + item.cantidad, 0)
  )
  const totalPrecio = computed(() =>
    items.value.reduce((total, item) => total + item.precio * item.cantidad, 0)
  )

  function agregar(producto, cantidad = 1) {
    const stock = Math.max(0, Number(producto.stock) || 0)
    if (stock === 0) return false

    const existente = items.value.find((item) => item.productoId === producto.id)

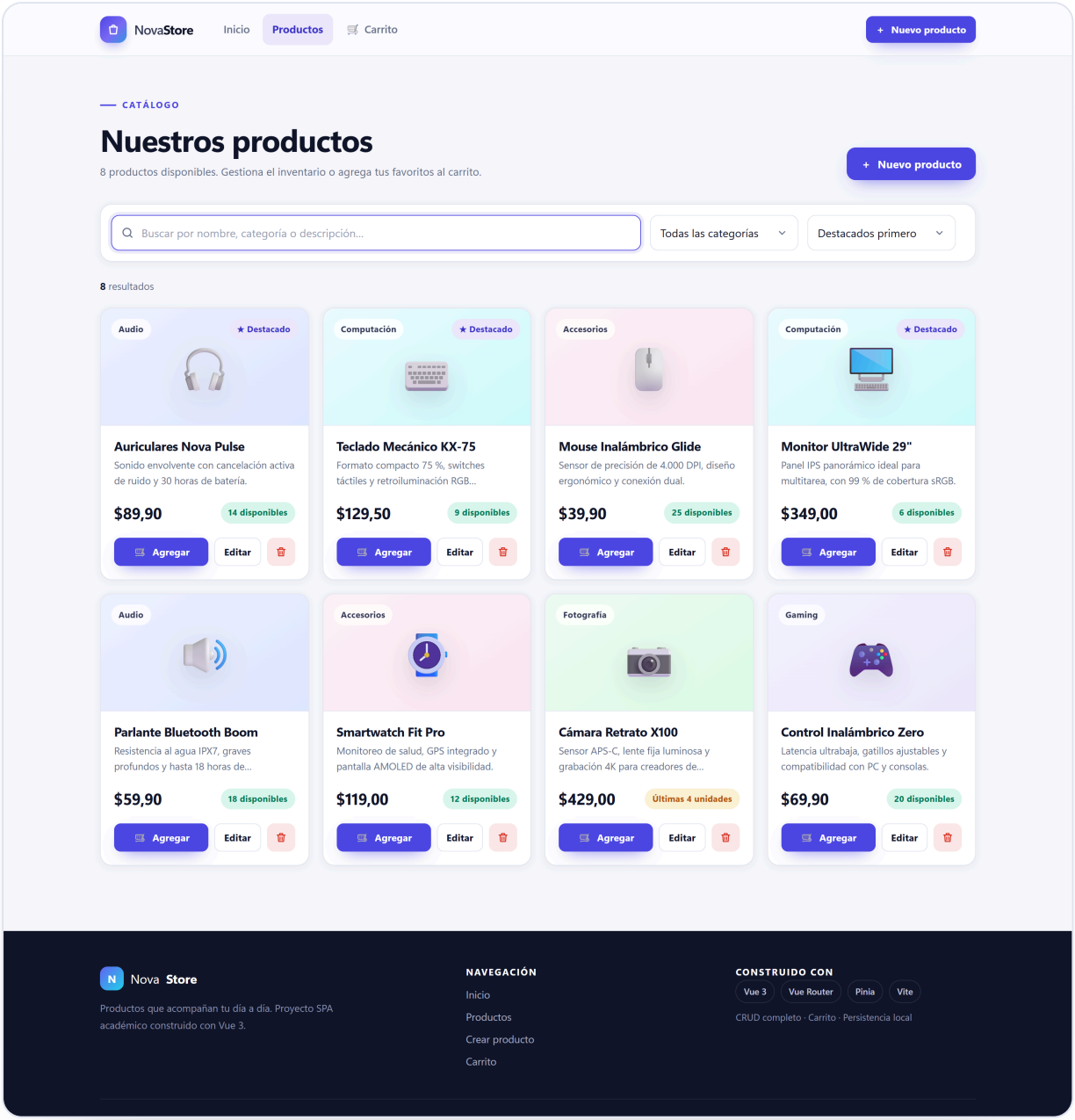
    if (existente) {
      existente.cantidad = Math.min(existente.cantidad + cantidad, stock)
      existente.precio = producto.precio
      existente.nombre = producto.nombre
      existente.icono = producto.icono
      existente.stock = stock
    } else {
      items.value.push({
        productoId: producto.id,
        nombre: producto.nombre,
        precio: producto.precio,
        icono: producto.icono,
        categoria: producto.categoria,
        stock,
        cantidad: Math.min(cantidad, stock)
      })
    }
    return true
  }
```

src/stores/cart.js · estado, totales y acción agregar

EVIDENCIA 05 · INTERFAZ DE USUARIO

Lista de productos con v-for

La lista de productos se genera con v-for sobre el resultado de la búsqueda y los filtros del store. Cada iteración utiliza una key estable basada en el id y delega la presentación en el componente ProductCard, que muestra nombre, precio, stock y los botones para editar, eliminar y agregar al carrito.



Ocho productos renderizados dinámicamente con v-for

Especificación cumplida
v-for con key sobre la lista filtrada de productos

EVIDENCIA 05 · CÓDIGO FUENTE

Renderizado de la lista con v-for

v-for recorre productosFiltrados y entrega cada producto como prop. La key permite a Vue reutilizar correctamente los nodos al filtrar o eliminar.

Nova Store · Evidencia de código fuente

Vue 3 · Pinia · Vue Router · Vite

src/views/ProductsView.vue — L. 118–125

```vue
<div v-if="productosFiltrados.length > 0" class="grilla-productos">
  <ProductCard
    v-for="producto in productosFiltrados"
    :key="producto.id"
    :producto="producto"
    @eliminar="pedirEliminar"
  />
</div>
```

src/components/ProductCard.vue — L. 49–60

```vue
<div class="producto__visual" :style="fondoIcono">
  <span class="producto__categoria badge">{{ producto.categoria }}</span>
  <span v-if="producto.destacado" class="producto__destacado badge badge--primary">
    ★ Destacado
  </span>
  <span class="producto__icono" aria-hidden="true">{{ producto.icono }}</span>
</div>

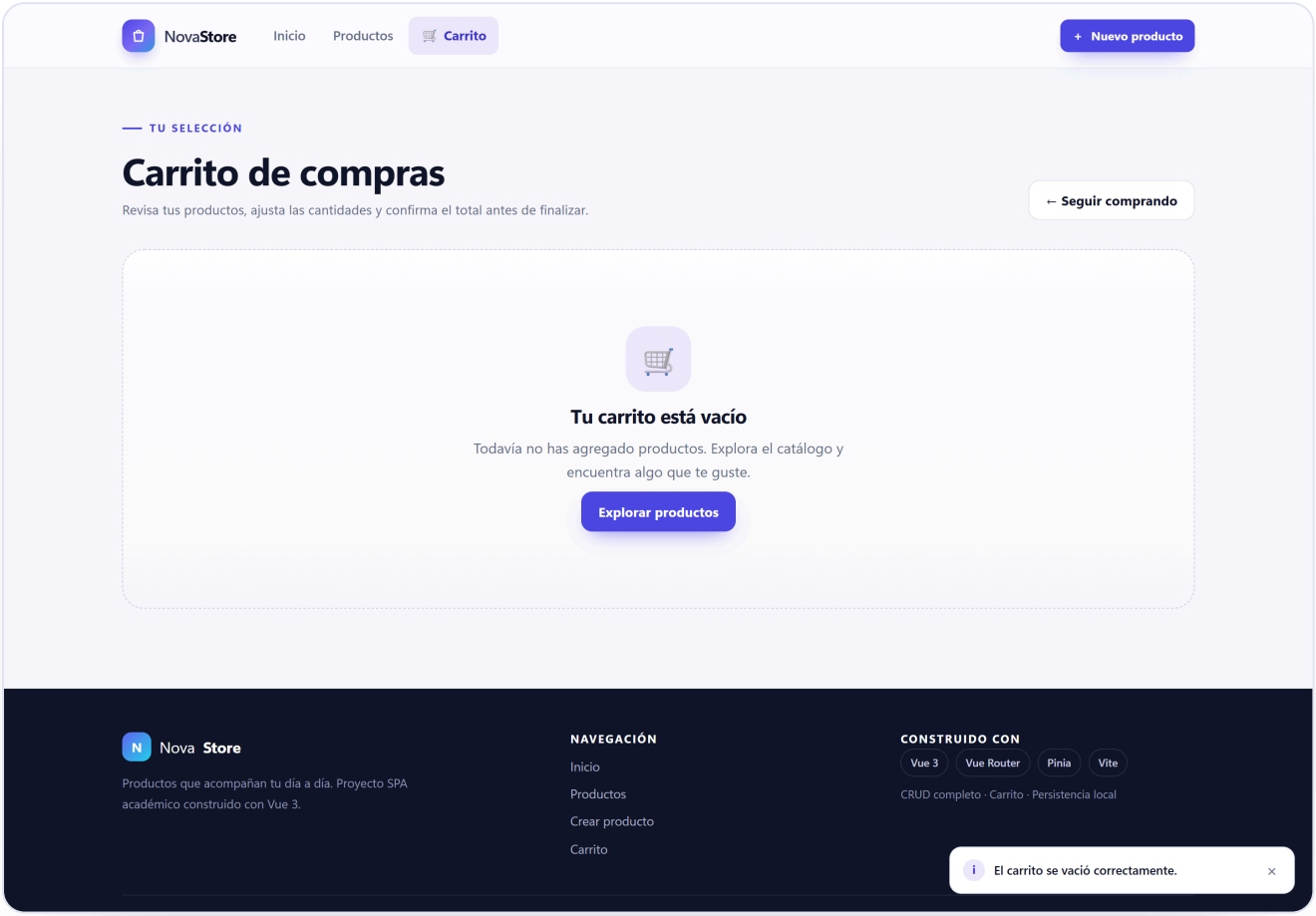
<div class="producto__cuerpo">
  <div class="producto__encabezado">
    <h3 class="producto__nombre">{{ producto.nombre }}</h3>
    <p class="producto__descripcion">{{ producto.descripcion }}</p>
```

src/views/ProductsView.vue y ProductCard.vue · bucle con key

EVIDENCIA 06 · INTERFAZ DE USUARIO

Estados condicionales con v-if, v-else-if y v-else

La interfaz utiliza directivas condicionales para representar los distintos estados de la aplicación: carrito vacío, resultados de búsqueda sin coincidencias, catálogo vacío, stock agotado, últimas unidades y errores de validación. Con v-if y v-else se alterna entre el estado vacío y el resumen del carrito, y con v-else-if se muestran mensajes de stock diferentes.



Estado vacío del carrito mostrado con v-if

Especificación cumplida
v-if, v-else-if y v-else para estados, stock y validaciones

EVIDENCIA 06 · CÓDIGO FUENTE

Estados condicionales en la interfaz

El mismo patrón se repite en la lista de productos y en los formularios, donde los mensajes de error solo se renderizan cuando existe una validación fallida.

Nova Store · Evidencia de código fuente

Vue 3 · Pinia · Vue Router · Vite

src/views/CartView.vue — L. 54–70

```vue
<div v-if="estaVacio" class="empty-state carrito-vacio">
  <span class="empty-state__icono" aria-hidden="true">🛒</span>
  <h2>{{ compraFinalizada ? '¡Gracias por tu compra!' : 'Tu carrito está vacío' }}</h2>
  <p v-if="compraFinalizada">
    Tu pedido simulado se registró correctamente. Puedes seguir explorando el catálogo.
  </p>
  <p v-else>
    Todavía no has agregado productos. Explora el catálogo y encuentra algo que te guste.
  </p>
  <RouterLink to="/productos" class="btn btn--primary">Explorar productos</RouterLink>
</div>

<div v-else class="carrito-layout">
  <section class="carrito-lista" aria-label="Productos en el carrito">
    <div class="carrito-lista__encabezado">
      <h2>Productos <span class="badge">{{ totalUnidades }} {{ totalUnidades === 1 ? 'unidad' : 'unidades' }}</sp
      <button type="button" class="btn btn--danger-soft btn--sm" @click="dialogoVaciar = true">
```

src/components/ProductCard.vue — L. 63–70

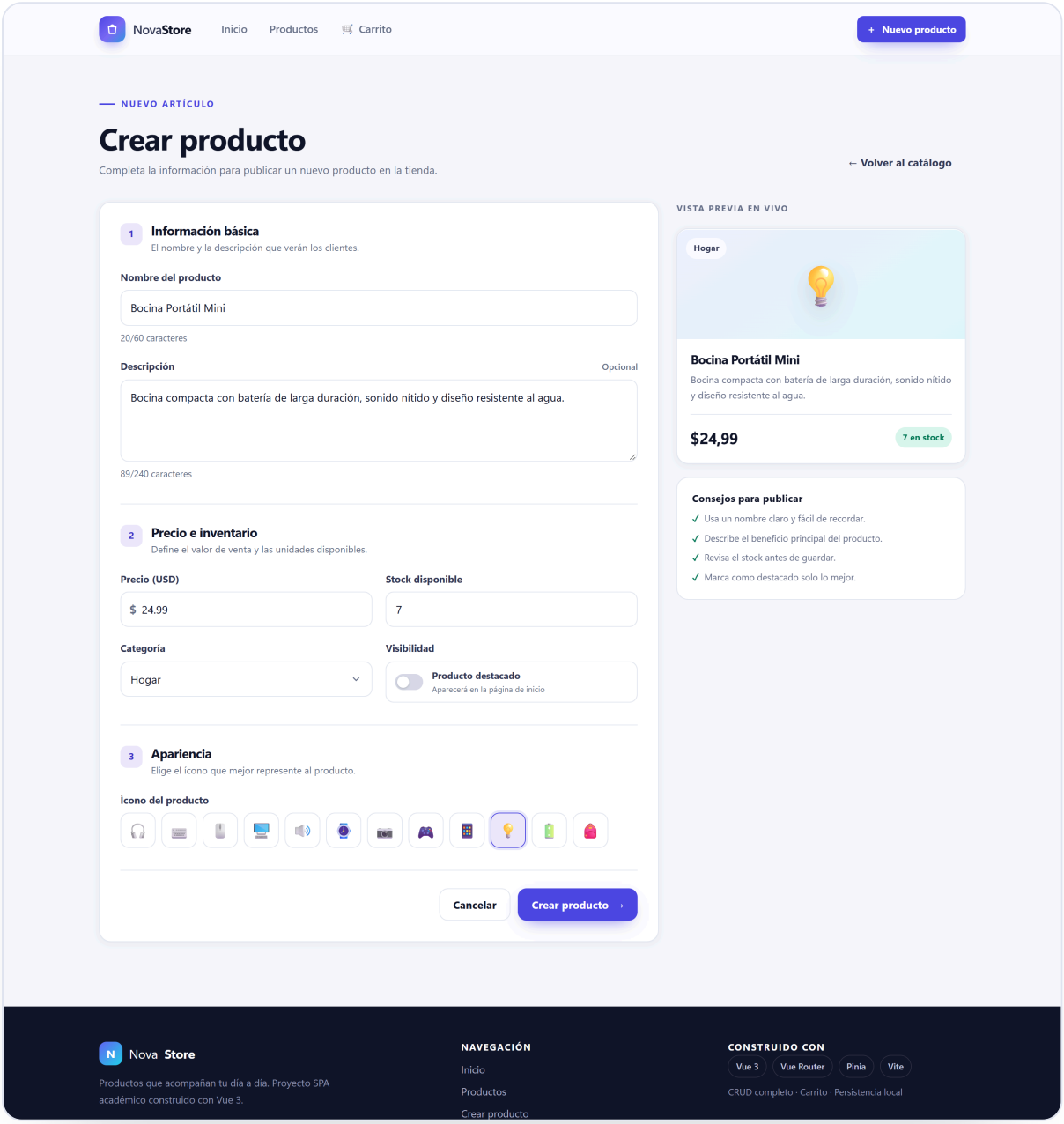
```vue
  <div class="producto__meta">
    <span class="price">{{ formatearPrecio(producto.precio) }}</span>
    <span v-if="agotado" class="badge badge--danger">Agotado</span>
    <span v-else-if="producto.stock <= 5" class="badge badge--warning">
      Últimas {{ producto.stock }} unidades
    </span>
    <span v-else class="badge badge--success">{{ producto.stock }} disponibles</span>
  </div>
```

src/views/CartView.vue y ProductCard.vue · v-if / v-else / v-else-if

EVIDENCIA 07 · INTERFAZ DE USUARIO

Formularios con v-model

El formulario enlaza todos sus campos con v-model: texto con .trim para nombre y descripción, .number para precio y stock, un select para la categoría y un checkbox para destacar el producto. La vista previa se actualiza en tiempo real y las validaciones muestran mensajes específicos antes de guardar.



Formulario de creación con enlace bidireccional y vista previa en vivo

Especificación cumplida
v-model.trim y v-model.number en el formulario de producto

EVIDENCIA 07 · CÓDIGO FUENTE

Enlace bidireccional con v-model

Los modificadores .trim y .number normalizan la entrada. El formulario se reutiliza para crear y editar mediante la ruta dinámica /productos/:id.

Nova Store · Evidencia de código fuente

Vue 3 · Pinia · Vue Router · Vite

src/views/ProductFormView.vue — L. 167–177 — TEXTO

```
167                <input
168                  id="nombre"
169                  v-foco
170                  v-model.trim="formulario.nombre"
171                  type="text"
172                  class="input"
173                  :class="{ 'input--error': errores.nombre }"
174                  placeholder="Ej. Auriculares Nova Pulse"
175                  maxlength="60"
176                  :aria-invalid="Boolean(errores.nombre)"
177                />
```

src/views/ProductFormView.vue — L. 215–241 — NÚMERO

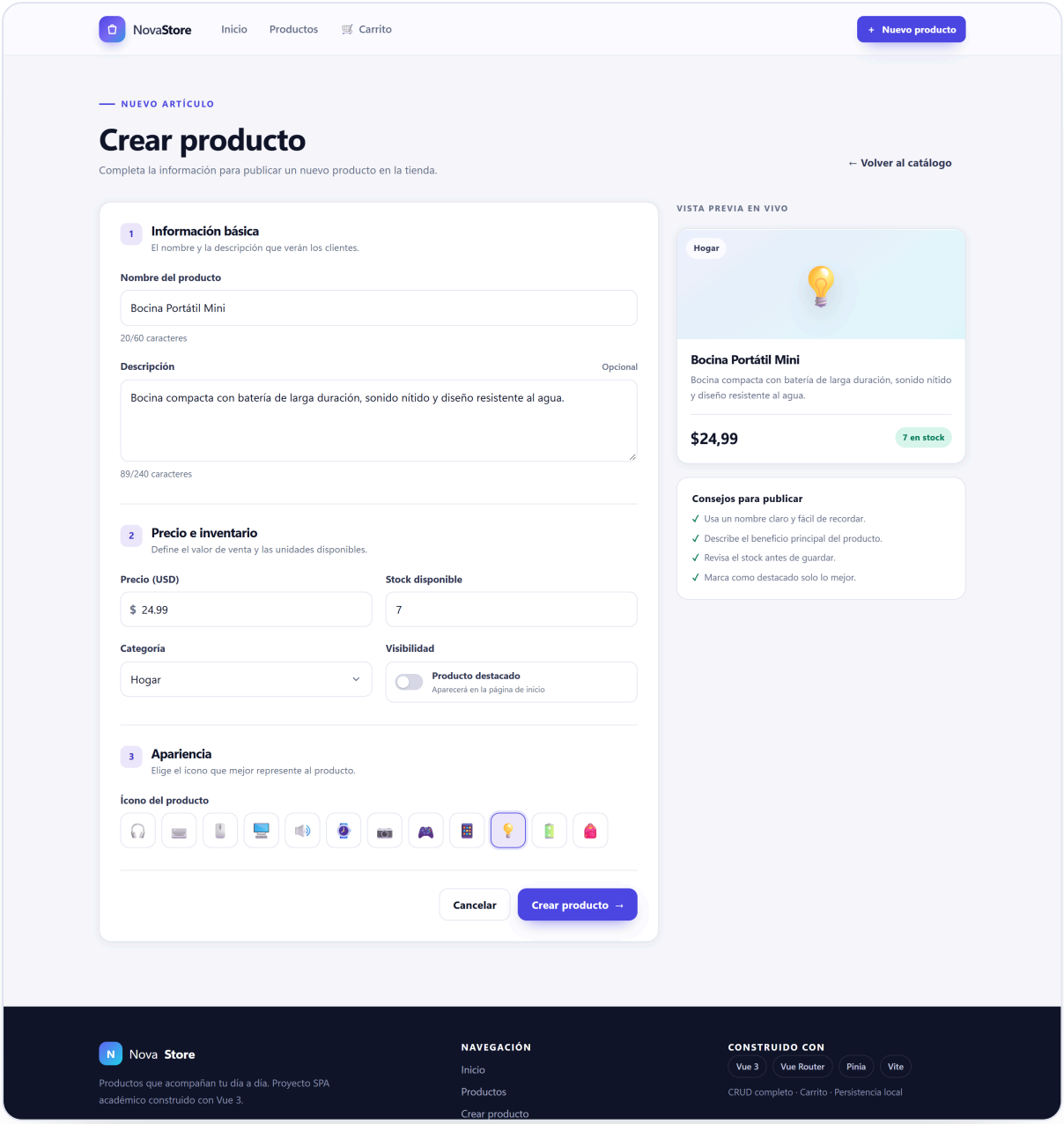
```
215                  <input
216                    id="precio"
217                    v-model.number="formulario.precio"
218                    type="number"
219                    class="input"
220                    :class="{ 'input--error': errores.precio }"
221                    placeholder="0.00"
222                    min="0.01"
223                    step="0.01"
224                    :aria-invalid="Boolean(errores.precio)"
225                  />
226                </div>
227                <p v-if="errores.precio" class="field__error" role="alert">{{ errores.precio }}</p>
228              </div>
229
230              <div class="field">
231                <label for="stock" class="label">Stock disponible</label>
232                <input
233                  id="stock"
234                  v-model.number="formulario.stock"
235                  type="number"
236                  class="input"
237                  :class="{ 'input--error': errores.stock }"
238                  min="0"
239                  step="1"
240                  :aria-invalid="Boolean(errores.stock)"
241                />
```

src/views/ProductFormView.vue · v-model.trim y v-model.number

EVIDENCIA 08 · INTERFAZ DE USUARIO

CRUD completo · Crear producto

Al enviar el formulario, la vista valida los datos y llama a la acción crear del store de productos. La acción normaliza los valores, calcula el siguiente id, agrega el producto al inicio del catálogo y redirige a la lista; el nuevo artículo aparece de inmediato y se conserva en localStorage.



Formulario diligenciado listo para registrar un nuevo producto

Especificación cumplida
Create · alta de productos desde el formulario

EVIDENCIA 08 · CÓDIGO FUENTE

Acción crear en el store de productos

La lógica de creación está en el store. El componente únicamente valida la entrada, invoca la acción y muestra una notificación de éxito.

Nova Store · Evidencia de código fuente

Vue 3 · Pinia · Vue Router · Vite

src/stores/products.js — L. 120–130 — NORMALIZAR

```
120 function normalizar(datos) {
121   return {
122     nombre: datos.nombre.trim(),
123     precio: Math.round((Number(datos.precio) || 0) * 100) / 100,
124     categoria: datos.categoria,
125     descripcion: (datos.descripcion || '').trim(),
126     stock: Math.max(0, Math.trunc(Number(datos.stock) || 0)),
127     icono: datos.icono || '📦',
128     destacado: Boolean(datos.destacado)
129   }
130 }
```

src/stores/products.js — L. 184–192 — CREAR

```
184   function crear(datos) {
185     const producto = {
186       ...normalizar(datos),
187       id: siguienteId(),
188       creadoEn: new Date().toISOString()
189     }
190     productos.value.unshift(producto)
191     return producto
192   }
```

src/views/ProductFormView.vue — L. 115–118 — FORMULARIO

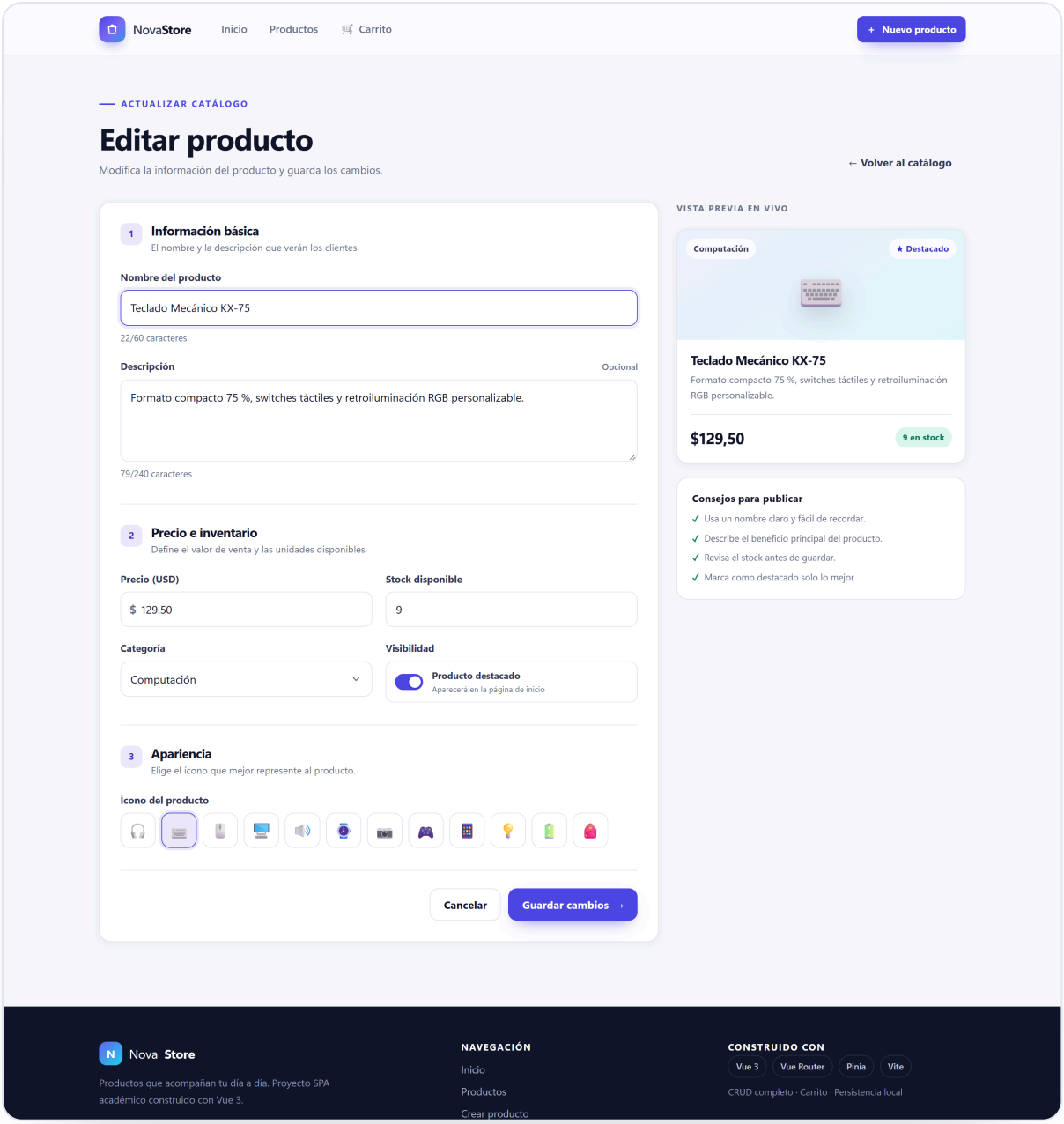
```
115   } else {
116     const creado = productosStore.crear(formulario)
117     notificar(`“${creado.nombre}” se agregó al catálogo.`, 'exito')
118   }
```

src/stores/products.js · normalizar y crear + envío del formulario

EVIDENCIA 09 · INTERFAZ DE USUARIO

CRUD completo · Editar producto

La ruta /productos/:id reutiliza el formulario en modo edición. Al montar la vista se busca el producto en el store y se copian sus valores al formulario; al guardar, la acción actualizar reemplaza el registro, mantiene su id y sincroniza nombre, precio y stock en el carrito.



Formulario de edición precargado con los datos del producto

Especificación cumplida
Update · edición de un producto existente

EVIDENCIA 09 · CÓDIGO FUENTE

Acción actualizar y carga del formulario

Si el id no existe se muestra un estado de producto no encontrado. La actualización se refleja en el catálogo y en el carrito sin recargar la página.

Nova Store · Evidencia de código fuente

Vue 3 · Pinia · Vue Router · Vite

src/views/ProductFormView.vue — L. 43–58 — CARGAR

```
43  const producto = productosStore.obtenerPorId(props.id)
44  if (!producto) {
45    noEncontrado.value = true
46    return
47  }
48
49  noEncontrado.value = false
50  Object.assign(formulario, {
51    nombre: producto.nombre,
52    precio: Number(producto.precio).toFixed(2),
53    categoria: producto.categoria,
54    descripcion: producto.descripcion,
55    stock: producto.stock,
56    icono: producto.icono,
57    destacado: producto.destacado
58  })
```

src/stores/products.js — L. 194–205 — ACTUALIZAR

```
194  function actualizar(id, datos) {
195    const indice = productos.value.findIndex((p) => p.id === Number(id))
196    if (indice === -1) return null
197
198    const actualizado = {
199      ...productos.value[indice],
200      ...normalizar(datos)
201    }
202    productos.value[indice] = actualizado
203    useCartStore().sincronizarProducto(actualizado)
204    return actualizado
205  }
```

src/views/ProductFormView.vue — L. 108–114 — GUARDAR

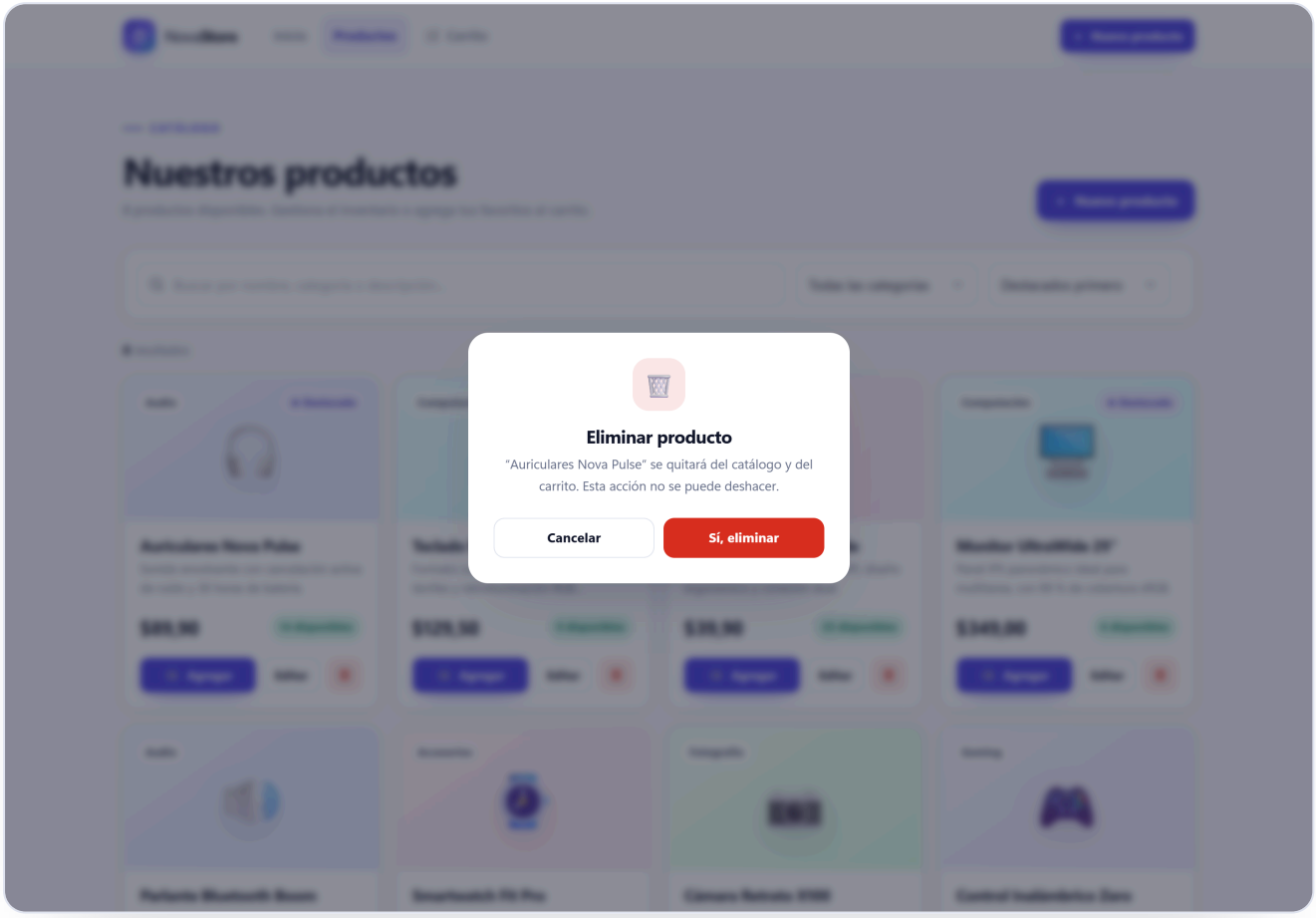
```
108  if (esEdicion.value) {
109    const actualizado = productosStore.actualizar(props.id, formulario)
110    if (!actualizado) {
111      notificar('No encontramos el producto que intentas editar.', 'error')
112      return
113    }
114    notificar(`“${actualizado.nombre}” se actualizó correctamente.`, 'exito')
```

src/views/ProductFormView.vue y src/stores/products.js · actualizar

EVIDENCIA 10 · INTERFAZ DE USUARIO

CRUD completo · Eliminar producto

Cada tarjeta incluye un botón de eliminación que abre un diálogo de confirmación para evitar borrados accidentales. Al confirmar, la vista invoca la acción eliminar del store, que quita el producto del catálogo y también lo retira del carrito si estaba agregado; después se muestra una notificación informativa.



Confirmación antes de eliminar un producto del catálogo

Especificación cumplida
Delete · eliminación con confirmación y limpieza del carrito

EVIDENCIA 10 · CÓDIGO FUENTE

Acción eliminar y confirmación

El componente solo gestiona la apertura del diálogo. La eliminación efectiva y la limpieza del carrito se ejecutan dentro de los stores de Pinia.

Nova Store · Evidencia de código fuente

Vue 3 · Pinia · Vue Router · Vite

src/views/ProductsView.vue — L. 36–45 — CONFIRMAR

```
36 function pedirEliminar(producto) {
37   productoAEliminar.value = producto
38 }
39
40 function confirmarEliminar() {
41   if (!productoAEliminar.value) return
42   productosStore.eliminar(productoAEliminar.value.id)
43   notificar(`“${productoAEliminar.value.nombre}” se eliminó del catálogo.`, 'info')
44   productoAEliminar.value = null
45 }
```

src/stores/products.js — L. 207–215 — ELIMINAR

```
207   function eliminar(id) {
208     const numero = Number(id)
209     const existe = productos.value.some((p) => p.id === numero)
210     if (!existe) return false
211
212     productos.value = productos.value.filter((p) => p.id !== numero)
213     useCartStore().eliminarProducto(numero)
214     return true
215   }
```

src/components/ProductCard.vue — L. 89–95 — BOTÓN

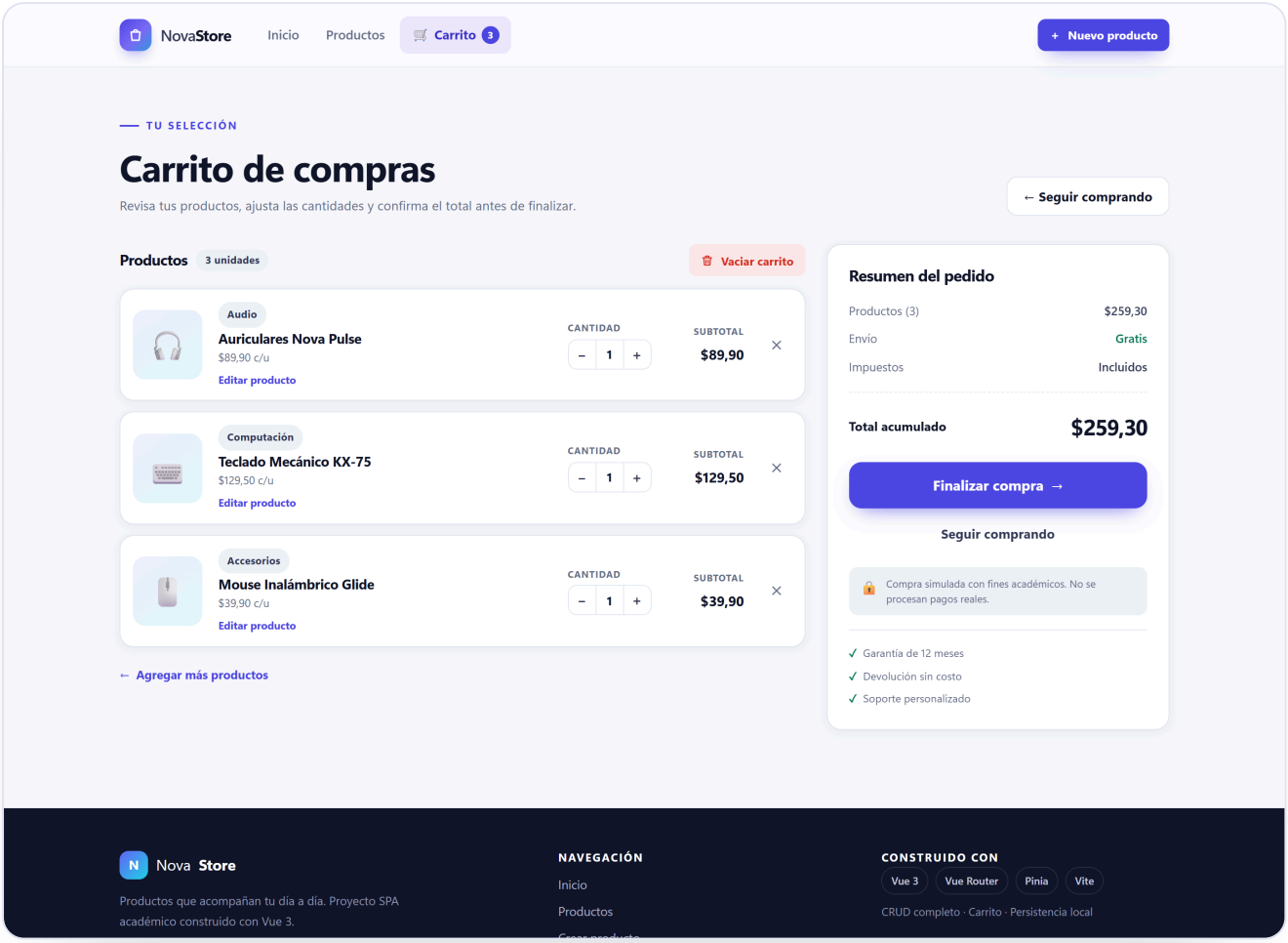
```
89         <button
90           type="button"
91           class="btn btn--danger-soft btn--icon"
92           :aria-label="`Eliminar ${producto.nombre}`"
93           title="Eliminar producto"
94           @click="$emit('eliminar', producto)"
95         >
```

src/views/ProductsView.vue y src/stores/products.js · eliminar

EVIDENCIA 11 · INTERFAZ DE USUARIO

Carrito de compras: cantidades, total y vaciado

La vista del carrito muestra cada producto, su precio unitario, la cantidad, el subtotal y el total acumulado. Los botones permiten incrementar o disminuir unidades sin superar el stock, eliminar un producto, vaciar el carrito con confirmación y finalizar una compra simulada; todas las operaciones se delegan al store.



Resumen del carrito con total acumulado y acciones disponibles

Especificación cumplida
Carrito · cantidades, subtotal, total acumulado y vaciado

EVIDENCIA 11 · CÓDIGO FUENTE

Operaciones y totales del carrito

El total se calcula como un getter a partir de precio por cantidad. Al llegar a cero unidades, disminuir elimina el artículo del carrito.

Nova Store · Evidencia de código fuente

Vue 3 · Pinia · Vue Router · Vite

src/stores/cart.js — L. 67–90 — OPERACIONES

```
67  function incrementar(productoId) {
68    const item = items.value.find((i) => i.productoId === productoId)
69    if (!item || item.cantidad >= item.stock) return false
70    item.cantidad += 1
71    return true
72  }
73
74  function disminuir(productoId) {
75    const item = items.value.find((i) => i.productoId === productoId)
76    if (!item) return
77    if (item.cantidad <= 1) {
78      quitar(productoId)
79    } else {
80      item.cantidad -= 1
81    }
82  }
83
84  function quitar(productoId) {
85    items.value = items.value.filter((item) => item.productoId !== productoId)
86  }
87
88  function vaciar() {
89    items.value = []
90  }
```

src/views/CartView.vue — L. 160–176 — TOTAL

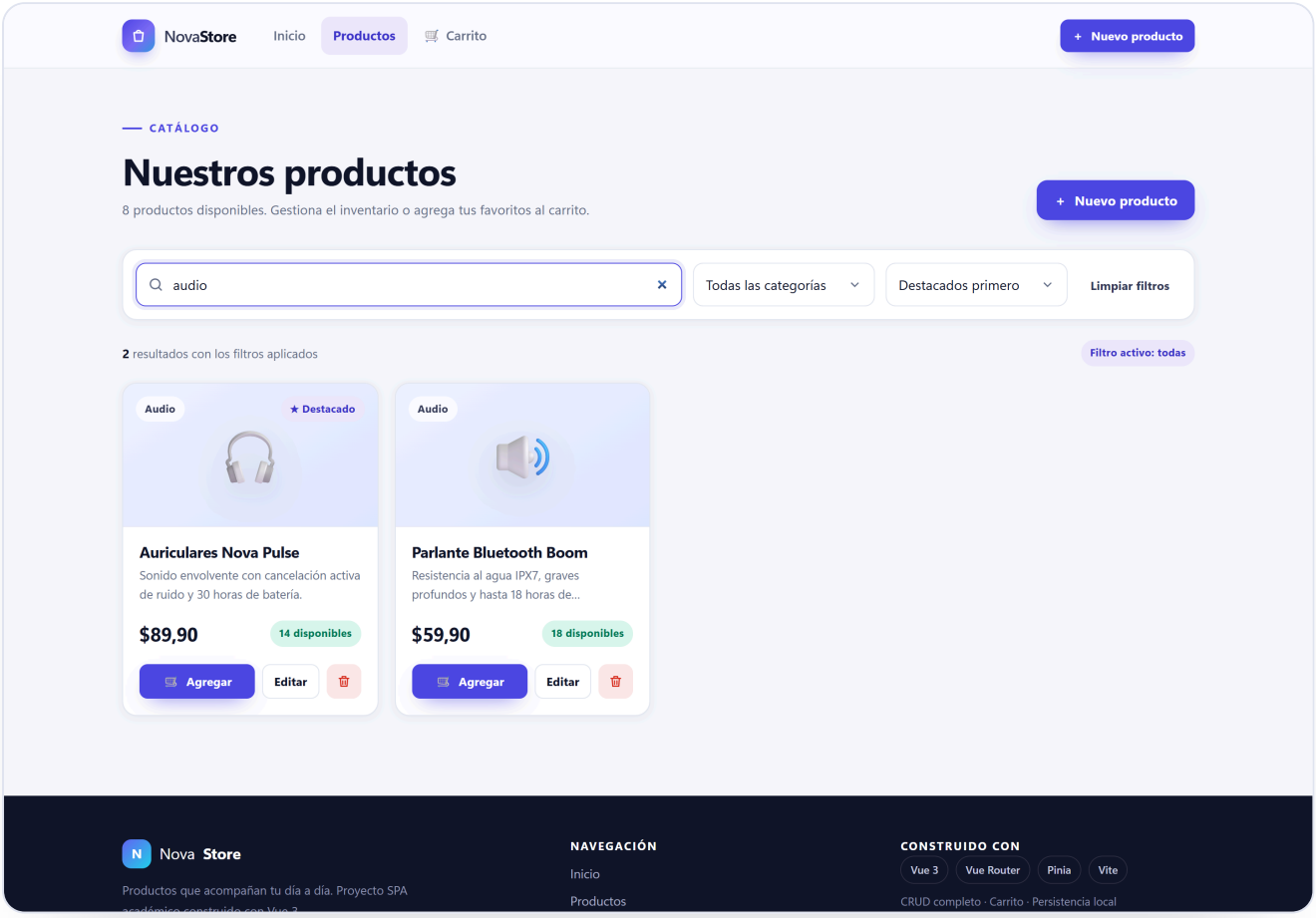
```
160         <div class="resumen__total">
161           <span>Total acumulado</span>
162           <strong>{{ formatearPrecio(totalPrecio) }}</strong>
163         </div>
164
165         <button type="button" class="btn btn--primary btn--lg btn--block" @click="finalizarCompra">
166           Finalizar compra
167           <span aria-hidden="true">→</span>
168         </button>
169         <RouterLink to="/productos" class="btn btn--ghost btn--block resumen__seguir">
170           Seguir comprando
171         </RouterLink>
172
173         <div class="resumen__nota">
174           <span aria-hidden="true">🔒</span>
175           <p>Compra simulada con fines académicos. No se procesan pagos reales.</p>
176         </div>
```

src/stores/cart.js · incrementar, disminuir, quitar y vaciar + total

EVIDENCIA 12 · INTERFAZ DE USUARIO

Persistencia local y directiva personalizada v-foco

Los stores sincronizan productos y carrito con localStorage, por lo que la información se conserva al recargar o cerrar el navegador. Además se registró la directiva personalizada v-foco, que coloca el cursor en el buscador y en el primer campo del formulario sin provocar desplazamientos inesperados.



Buscador enfocado automáticamente con la directiva v-foco

Especificación cumplida
Persistencia en localStorage y directiva v-foco

EVIDENCIA 12 · CÓDIGO FUENTE

Persistencia y directiva personalizada

Los datos se leen al inicializar cada store y se escriben con watch en modo profundo. La directiva se registra globalmente en main.js.

Nova Store · Evidencia de código fuente

Vue 3 · Pinia · Vue Router · Vite

src/stores/products.js — L. 101–110 — CARGA

```
function cargarProductos() {
  try {
    const guardado = localStorage.getItem(CLAVE_PRODUCTOS)
    if (!guardado) return structuredClone(PRODUCTOS_INICIALES)
    const datos = JSON.parse(guardado)
    return Array.isArray(datos) ? datos : structuredClone(PRODUCTOS_INICIALES)
  } catch {
    return structuredClone(PRODUCTOS_INICIALES)
  }
}
```

src/stores/cart.js — L. 25–31 — WATCH

```
  watch(
    items,
    (valor) => {
      localStorage.setItem(CLAVE_CARRITO, JSON.stringify(valor))
    },
    { deep: true }
  )
```

src/directives/foco.js — L. 1–12 — DIRECTIVA

```
/**
 * Directiva personalizada v-foco.
 * Coloca el cursor en el campo cuando el elemento se monta en el DOM.
 * Se usa en el buscador de productos y en el primer campo del formulario.
 */
export const vFoco = {
  mounted(el) {
    requestAnimationFrame(() => el.focus({ preventScroll: true }))
  }
}

```

Carga desde localStorage, watch de persistencia y directiva v-foco

EVIDENCIA 13 · INTERFAZ DE USUARIO

Interfaz legible, estilos y diseño responsive

La interfaz se construyó con CSS propio, variables de diseño, componentes reutilizables y puntos de quiebre para escritorio, tableta y móvil. El menú, las tarjetas, los formularios y el carrito se reorganizan en una columna, manteniendo jerarquía visual, contrastes accesibles y objetivos táctiles cómodos.



Vista móvil de la página de inicio (390 × 844 px)

Especificación cumplida
CSS responsive con puntos de quiebre para móvil

EVIDENCIA 13 · CÓDIGO FUENTE

Variables CSS y diseño responsive

Las variables centralizan colores, radios y sombras. Los media queries adaptan la navegación, las cuadrículas y los formularios a pantallas pequeñas.

Nova Store · Evidencia de código fuente

Vue 3 · Pinia · Vue Router · Vite

src/assets/main.css — L. 1–24 — VARIABLES

```css
:root {
  color-scheme: light;
  --bg: #f5f6fb;
  --surface: #ffffff;
  --surface-soft: #eef0f9;
  --ink: #11152a;
  --ink-soft: #343b58;
  --muted: #68708a;
  --border: #e4e7f1;
  --primary: #4f46e5;
  --primary-dark: #3f36c9;
  --primary-soft: #eceafe;
  --accent: #06b6d4;
  --success: #047857;
  --success-soft: #d9f7ec;
  --danger: #d92d20;
  --danger-soft: #fde8e7;
  --warning: #b45309;
  --warning-soft: #fdf0d5;
  --radius-sm: 10px;
  --radius: 16px;
  --radius-lg: 24px;
  --shadow-sm: 0 2px 8px rgba(17, 21, 42, 0.05);
  --shadow: 0 16px 40px -20px rgba(24, 30, 66, 0.22);
```

src/views/HomeView.vue — L. 646–663 — RESPONSIVE

```css
@media (max-width: 640px) {
  .hero__visual {
    min-height: 360px;
  }

  .hero__blob {
    width: 320px;
    height: 320px;
  }

  .hero__flotante {
    display: none;
  }

  .estadisticas {
    grid-template-columns: 1fr;
    gap: 12px;
    padding: 18px;
```

src/assets/main.css y HomeView.vue · variables y media queries